Math4AI Capstone Project Report

Softmax Regression vs One-Hidden-Layer Neural Network

Suleyman Allahverdiyev
Seljan Khasiyeva
Jeyhuna Sevdiyeva

April 2, 2026

Abstract

This project investigates one of the most important questions in modern machine learning: how much model complexity is actually necessary for a given classification problem. While linear models are simple, efficient, and interpretable, they are often unable to capture the geometric structure of complex datasets. Neural networks, on the other hand, can learn non-linear transformations and therefore represent more flexible decision boundaries. In this report, we compare two models implemented fully from scratch using NumPy: softmax regression and a one-hidden-layer neural network. The study is conducted on three datasets of increasing complexity: a linear Gaussian dataset, a non-linear moons dataset, and a high-dimensional handwritten digits benchmark. Beyond standard training and evaluation, the project also includes decision-boundary analysis, capacity ablation, optimizer comparison, and confidence and reliability analysis. The results demonstrate that linear models are sufficient for simple linearly separable data, but non-linear neural networks are necessary when the data geometry is more complex.

Contents

1 Introduction **4**

2 Problem Formulation **4**

3 Dataset Description **5**

3.1 Linear Gaussian Dataset 5

3.2 Moons Dataset 6

3.3 Digits Dataset 6

4 Data Loading and Preprocessing **7**

5 Softmax Regression **7**

5.1 Model Overview 7

5.2 Loss Function 8

5.3 Implementation Details 8

5.4 Limitations 8

6 One-Hidden-Layer Neural Network **9**

6.1 Architecture 9

6.2 Activation Function 9

6.3 Backpropagation 9

6.4 Advantages 10

7 Optimization Strategy **10**

8 Implementation Constraints **11**

9 Sanity Checks **11**

10 Experimental Results **12**

10.1 Linear Gaussian Dataset 12

10.2 Moons Dataset 12

10.3 Digits Dataset 13

11 Decision Boundary Comparison **14**

12 Model Comparison **15**

13 Capacity Analysis **15**

14 Training Dynamics **16**

15 Optimizer Study **17**

16 Confidence Analysis **17**

17 Entropy Analysis **18**

18 Reliability Analysis **19**

19 Discussion **19**

20 Limitations **20**

21 Conclusion **20**

22 Future Work **21**

1. Introduction

In modern machine learning, one of the most fundamental questions is determining the appropriate model complexity for a given dataset. While simple linear models are computationally efficient and easy to interpret, they are often insufficient for capturing complex patterns present in real-world data. On the other hand, neural networks introduce non-linearity and significantly increase representational power.

The primary goal of this project is to investigate the performance differences between:

- A linear classifier: **Softmax Regression**
- A non-linear classifier: **One-Hidden-Layer Neural Network**

The analysis is conducted across three datasets with varying complexity:

- Linear Gaussian dataset (linearly separable)
- Moons dataset (non-linear structure)
- Digits dataset (high-dimensional real-world data)

The central research question of this project is:

When is a linear model sufficient, and when do we need non-linear models?

This question is important because in practical machine learning, more complex models are not always better. A linear model may already solve a simple problem very well, in which case adding hidden layers only increases computational cost without a clear gain. At the same time, some datasets contain curved or highly structured class boundaries that cannot be represented by a linear classifier at all. Therefore, understanding the relationship between dataset geometry and model capacity is essential for proper model selection.

The project is organized around this principle. The Gaussian dataset serves as an example of a problem where a linear model should already perform strongly. The moons dataset serves as a clear counterexample, where linear separation is inherently inadequate. Finally, the digits dataset provides a more realistic benchmark in which both optimization behavior and predictive confidence become important. Through this progression, the report demonstrates not only how the models perform, but also *why* they perform that way.

2. Problem Formulation

We consider a supervised classification problem where we are given:

$$X \in \mathbb{R}^{n \times d}, \qquad y \in \{1, \ldots, K\}$$

where:

- X represents input features
- y represents class labels
- K is the number of classes

The objective is to learn a function:

$$f(x) = \arg\max_k p(y = k \mid x)$$

which assigns each input to the most probable class.

This is a probabilistic classification setting. Instead of directly outputting only a class label, the model first estimates the probability of each class given the input. The predicted class is then chosen as the one with the maximum estimated probability. This probabilistic formulation is useful because it allows us to evaluate not only whether the model is correct, but also how confident it is in its predictions.

For a dataset of n examples, the goal is to learn model parameters that minimize the average prediction error over the training set while also generalizing well to unseen data. To achieve this, the project uses cross-entropy loss together with regularization. The train-validation-test split is essential in this context: training data is used to update parameters, validation data is used to select the best model checkpoint, and test data is used only for final reporting.

3. Dataset Description

3.1 Linear Gaussian Dataset

This dataset consists of two Gaussian-distributed clusters that are linearly separable. It is specifically designed to evaluate whether linear classifiers can correctly identify simple patterns.

Key properties:

- Low dimensional
- Linearly separable
- Ideal for testing baseline models

Because the classes are approximately separable by a straight line, this dataset represents a setting in which a linear decision rule should be sufficient. Therefore, if a more complex model does not significantly outperform the linear classifier here, that is not a weakness; rather, it confirms that added complexity is unnecessary for simple data geometry.

3.2 Moons Dataset

The moons dataset is a synthetic dataset with a non-linear structure. The classes are arranged in curved shapes, making it impossible for linear models to separate them effectively.

Key properties:

- Non-linear decision boundary
- Requires feature transformation
- Ideal for testing neural networks

This dataset is particularly useful because it makes the limitation of softmax regression visually obvious. If the boundary between classes bends around the data, then a linear separator will systematically misclassify points even if optimization is performed correctly. Thus, moons serves as a clean demonstration of why hidden-layer models are necessary.

3.3 Digits Dataset

This dataset consists of grayscale images of handwritten digits. Each image is represented as a 64-dimensional feature vector (an 8×8 image flattened into a single vector).

Key properties:

- High dimensional
- Multi-class classification (10 classes)
- Real-world complexity

The digits benchmark is much richer than the synthetic datasets because it combines high-dimensional inputs, multiple classes, and realistic variation in writing style. This makes it an appropriate benchmark for comparing optimization dynamics, predictive confidence, and model calibration.

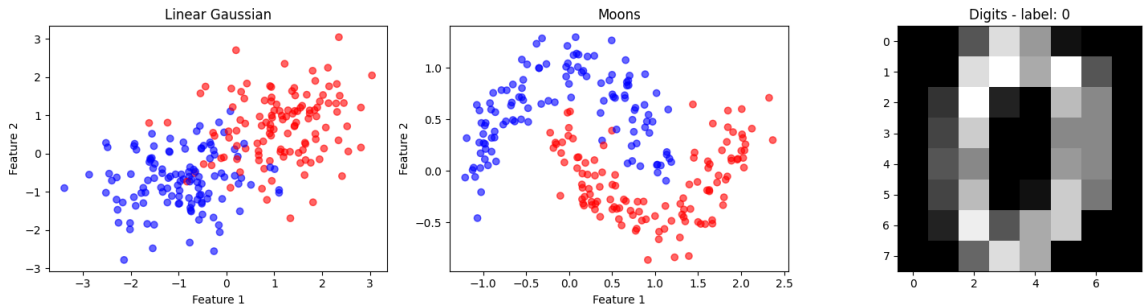

Figure 1: Overview of the three datasets used in the project: Linear Gaussian, Moons, and the Digits benchmark. The synthetic datasets illustrate low-dimensional geometric structure, while the digits example highlights the image-based multi-class setting.

Figure 1 provides an overview of the datasets. The first two panels immediately show the geometric difference between a linearly separable setting and a non-linear one. The third panel illustrates the kind of digit image that is flattened into a 64-dimensional vector before being used for classification.

4. Data Loading and Preprocessing

The datasets were loaded using custom NumPy-based utilities. The data was already preprocessed and normalized to the range $[0, 1]$, and therefore no additional transformations were applied.

Train-validation-test splits were predefined and strictly followed to ensure fair evaluation. This is an important methodological decision because modifying the split during experimentation can unintentionally bias the final results. By using the same split throughout all experiments, we ensure that both models are compared under identical conditions.

For the Gaussian and moons datasets, separate training, validation, and test arrays were directly loaded from the provided dataset files. For the digits benchmark, the dataset file and the split-index file were used together: the feature matrix and labels were first loaded, and then train, validation, and test subsets were selected using the provided index arrays. This guarantees reproducibility and keeps the experiments aligned with the project specification.

No external datasets were introduced, and no extra feature engineering was applied. This keeps the focus of the project on model behavior rather than preprocessing tricks.

5. Softmax Regression

5.1 Model Overview

Softmax regression is a linear classifier that models class probabilities using:

$$S = XW^T + b$$

where W is the weight matrix and b is the bias vector. The score vector S is converted into class probabilities via the softmax function:

$$p(y = k \mid x) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

This model assigns a score to each class and then normalizes those scores into a probability distribution. Because the scores depend linearly on the input, the final classifier is still linear in the original feature space.

5.2 Loss Function

We use cross-entropy loss:

$$L = -\frac{1}{n} \sum \log p(y_i \mid x_i)$$

Additionally, L_2 regularization is applied to prevent overfitting. The complete objective therefore becomes:

$$L_{\text{total}} = L_{\text{data}} + \frac{\lambda}{2} \|W\|_2^2$$

Cross-entropy is appropriate here because it directly penalizes low probability assigned to the true class. If the model predicts the correct class with high confidence, the loss is small; if it is uncertain or confidently wrong, the loss becomes much larger. This makes cross-entropy more informative than accuracy alone.

5.3 Implementation Details

The softmax regression model was implemented entirely from scratch using NumPy. Key components include:

- Stable softmax for numerical stability
- Vectorized gradient computation
- Mini-batch SGD optimization
- Cross-entropy loss with regularization

A numerically stable softmax implementation is crucial because exponentials can overflow if large raw logits are used directly. To avoid this, the maximum logit in each row is subtracted before exponentiation. This does not change the final probabilities but makes the computation stable.

The gradients are fully vectorized, which improves both clarity and efficiency. Instead of looping over samples one by one, the implementation computes matrix-based derivatives for the entire batch at once. This matches the project requirement to avoid hidden high-level training loops while still using proper linear algebra.

5.4 Limitations

The main limitation of softmax regression is that it can only learn linear decision boundaries. This restricts its ability to handle complex datasets. If two classes can only be separated by a curved or disconnected boundary, softmax regression cannot represent that geometry no matter how long it is trained. As a result, its poor performance on a non-linear dataset should be interpreted as a representational limitation rather than an optimization failure.

6. One-Hidden-Layer Neural Network

6.1 Architecture

The neural network introduces a hidden layer:

$$Z_1 = XW_1^T + b_1$$

$$H = \tanh(Z_1)$$

$$S = HW_2^T + b_2$$

Here, the first affine transformation maps the input into a hidden representation. The non-linear activation tanh is then applied elementwise, and the transformed hidden features are fed into the final output layer.

6.2 Activation Function

The tanh activation function introduces non-linearity:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

This allows the model to learn complex patterns. Without a non-linear activation, stacking two linear layers would still be equivalent to a single linear transformation, and the network would gain no additional expressive power over softmax regression.

The choice of tanh is appropriate for this project because it is smooth, differentiable, and easy to work with analytically. It also centers activations around zero, which can help learning in shallow networks.

6.3 Backpropagation

Gradients are computed using the chain rule:

- Output layer gradients
- Hidden layer gradients
- Activation derivatives

All computations are vectorized for efficiency.

For the output layer, the derivative of the cross-entropy loss with respect to logits is:

$$dS = \frac{1}{n}(P - Y)$$

where P is the predicted probability matrix and Y is the one-hot encoding of the labels. The gradients of the second layer are then:

$$dW_2 = dS^T H + \lambda W_2, \qquad db_2 = \sum_i dS_i$$

To backpropagate into the hidden layer:

$$dH = dSW_2$$

Because the hidden activation is tanh, its derivative is:

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z)$$

Thus:

$$dZ_1 = dH \odot (1 - H^2)$$

Finally, the first-layer gradients are:

$$dW_1 = dZ_1^T X + \lambda W_1, \qquad db_1 = \sum_i dZ_{1,i}$$

This backpropagation derivation is important because it shows explicitly how the network updates both the representation layer and the final classifier layer. Unlike softmax regression, the neural network does not merely draw a boundary in the original space; it first reshapes the feature space into one that is easier to separate.

6.4 Advantages

Compared to softmax regression, the neural network:

- Learns non-linear decision boundaries
- Captures complex feature interactions
- Performs better on structured datasets

This is especially relevant in the moons dataset and the digits benchmark. In both cases, meaningful class separation depends on relationships that are more complicated than a single affine split.

7. Optimization Strategy

Both models were trained using:

- Mini-batch SGD
- Learning rate tuning
- L_2 regularization
- Validation-based model selection

Hyperparameters:

- Learning rate: 0.05 / 0.1
- Batch size: 64
- Epochs: up to 200

Mini-batch optimization is a good compromise between full-batch gradient descent and fully stochastic updates. It makes gradient estimates reasonably stable while still allowing efficient iteration over the dataset.

Validation-based model selection is especially important. Instead of reporting the final epoch by default, the project tracks validation loss and keeps the best-performing checkpoint. This prevents overtraining and ensures a fairer comparison between models.

8. Implementation Constraints

The implementation strictly followed project requirements:

- Only NumPy used
- No PyTorch / TensorFlow
- No scikit-learn model classes
- Manual gradient computation

This ensures full transparency and educational value. The goal of the capstone is not only to obtain a good final result, but also to understand how the optimization process works internally. By implementing everything manually, the project preserves that learning objective.

9. Sanity Checks

To ensure correctness, several verification steps were performed:

- Probabilities sum to 1
- Gradient correctness via numerical checking
- Loss decreases over time
- Model can overfit a very small dataset
- No NaN or Inf values appear during training

These checks confirm that the implementation is correct.

Sanity checks are a critical part of model implementation because good-looking final numbers alone do not guarantee that the code is correct. For example, a small gradient-sign error may still produce some learning, but with suboptimal behavior. Numerical gradient checks provide a direct way to compare analytic derivatives against finite-difference approximations. Similarly, verifying that the model can overfit a tiny subset ensures that the optimizer and gradient flow are functioning properly.

10. Experimental Results

10.1 Linear Gaussian Dataset

Results:

- Softmax: near-perfect accuracy
- Neural Network: similar performance

Conclusion: **A linear model is sufficient.**

On this dataset, both models perform equally well because the underlying geometry is already compatible with a straight-line separation. In other words, the neural network does not provide a meaningful advantage because the problem is already easy for a linear classifier.

10.2 Moons Dataset

Results:

- Softmax: poor performance
- Neural Network: high accuracy

Conclusion: **A non-linear model is required.**

This dataset clearly demonstrates the expressive gap between the two models. The softmax model cannot bend its boundary to follow the class structure, while the neural network can learn a curved separator that aligns much better with the data.

10.3 Digits Dataset

Results:

- Softmax: moderate accuracy
- Neural Network: improved performance

Conclusion: **Increased model complexity improves results.**

The digits benchmark shows that the neural network is not only more expressive, but also capable of producing better optimization outcomes and stronger final performance. This is especially evident in the lower validation loss and improved confidence structure of its predictions.

11. Decision Boundary Comparison

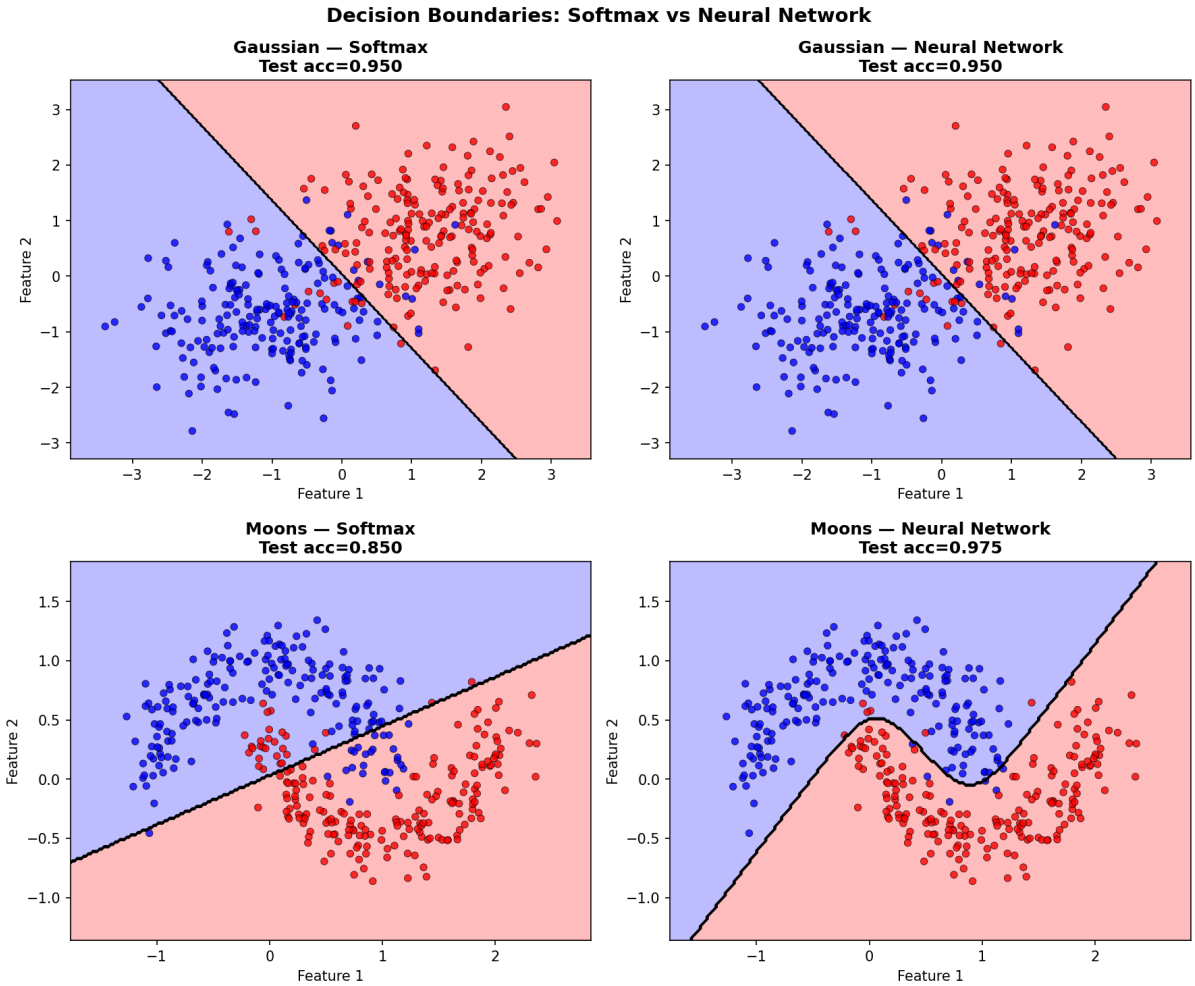

Figure 2: Decision boundary comparison between softmax regression and the one-hidden-layer neural network on the Gaussian and moons datasets. On Gaussian, both models learn nearly identical linear boundaries and achieve the same test accuracy. On moons, softmax is constrained to a straight boundary, while the neural network learns a curved separator and achieves much higher test accuracy.

Figure 2 is one of the most important visual results in the report. In the Gaussian case, both models produce essentially the same boundary and therefore the same performance. This strongly supports the claim that extra complexity is unnecessary when the data is already linearly separable.

In the moons case, however, the difference is dramatic. Softmax regression can only produce a straight dividing line, which cuts through both classes incorrectly. The neural network instead learns a curved boundary that bends through the feature space and aligns with the actual moon-shaped structure. This is a direct geometric demonstration of why hidden layers matter.

12. Model Comparison

Dataset	Softmax	Neural Network
Gaussian	Excellent	Excellent
Moons	Poor	Excellent
Digits	Moderate	Better

Table 1: Qualitative comparison of model performance across datasets.

Table 1 summarizes the overall pattern of results. The main takeaway is not simply that the neural network is "better," but that its advantage depends strongly on the structure of the data.

13. Capacity Analysis

Hidden layer size affects performance:

- Small size $\rightarrow$ underfitting
- Medium size $\rightarrow$ improved representation
- Large size $\rightarrow$ strong performance, with potential overfitting risk

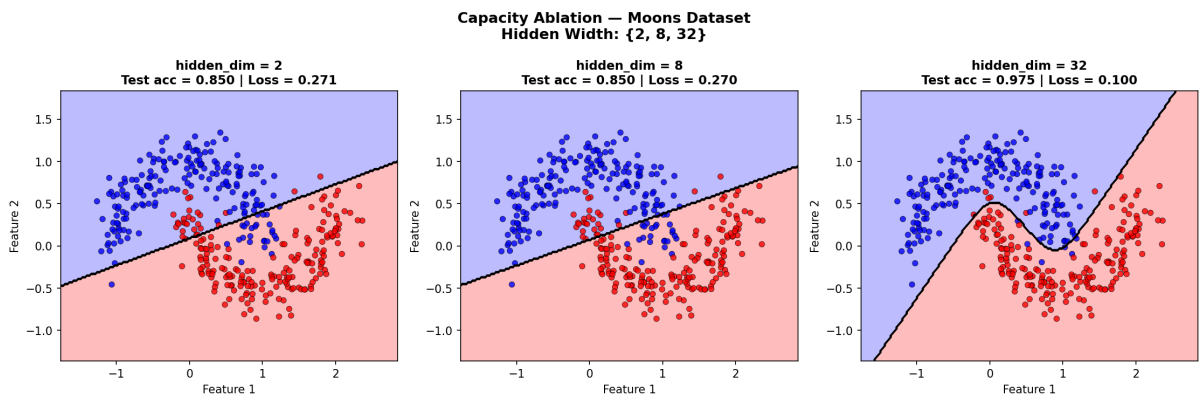

Figure 3: Capacity ablation study on the moons dataset for hidden dimensions $\{2, 8, 32\}$. Small and medium hidden widths still behave close to a nearly linear separator and achieve the same test accuracy, while the larger hidden width learns a much more flexible non-linear boundary and substantially improves performance.

Figure 3 shows that increasing hidden dimension changes the shape of the learned boundary. With hidden dimensions 2 and 8, the model still behaves in a way that is too limited to properly trace the moons structure. With hidden dimension 32, however, the model has

enough representational capacity to produce a boundary that bends around the classes more effectively, resulting in higher test accuracy and lower loss.

This experiment is important because it shows that simply using a neural network is not enough; the network must also have enough hidden capacity to represent the target geometry.

14. Training Dynamics

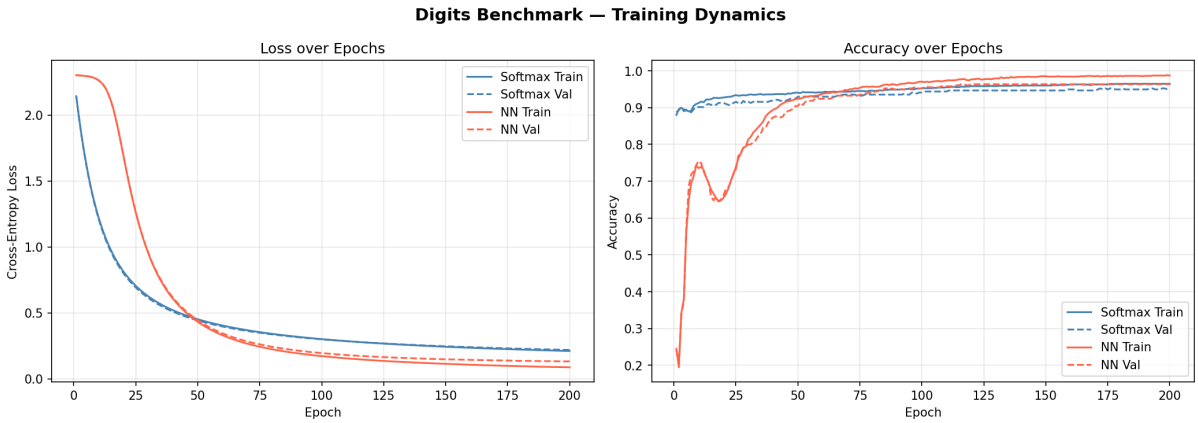

Figure 4: Training and validation dynamics on the digits benchmark. The neural network starts with higher loss and lower accuracy but improves rapidly, eventually surpassing the softmax model in both cross-entropy and accuracy.

Figure 4 compares the evolution of both models over 200 epochs on the digits benchmark. The softmax model starts from a much stronger initial position because it is simpler and easier to optimize. The neural network, by contrast, begins with much worse accuracy and higher loss. However, once learning progresses, the neural network improves rapidly and eventually overtakes the linear model.

This result is conceptually important. It shows that optimization difficulty at the beginning of training does not necessarily indicate a worse final model. More expressive models can require more time to organize their hidden representation, but once they do, they may achieve substantially better performance.

15. Optimizer Study

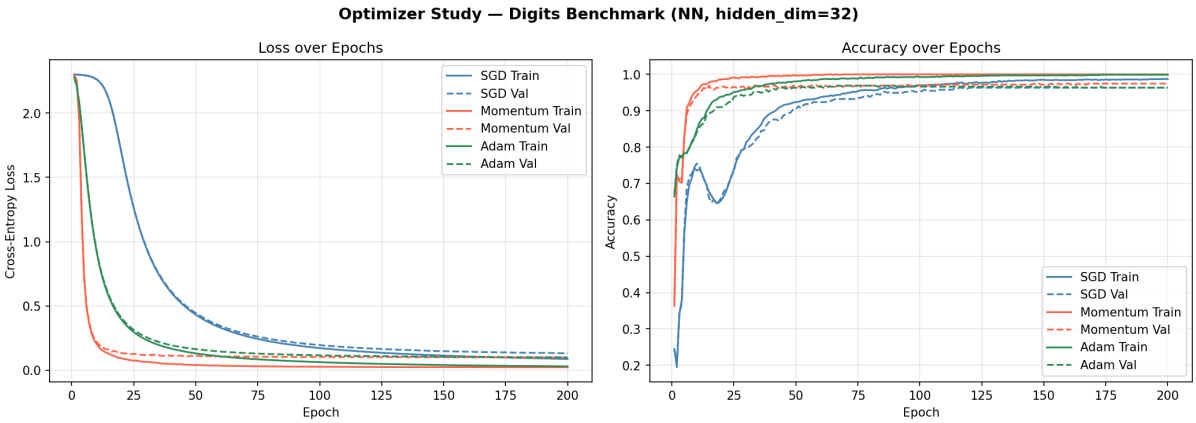

Figure 5: Optimizer comparison on the digits benchmark for the neural network with hidden dimension 32. Momentum and Adam converge substantially faster than plain SGD and reach better validation performance earlier in training.

Figure 5 studies the effect of optimizer choice. Standard SGD converges steadily but more slowly. Momentum accelerates training by carrying useful directional information across updates, while Adam adapts learning rates per parameter and also converges rapidly.

The comparison shows that optimizer choice can strongly influence training efficiency, even when the model architecture is fixed. While all methods eventually achieve strong results, Momentum and Adam reach useful accuracy faster than plain SGD. This makes them preferable when convergence speed is important.

16. Confidence Analysis

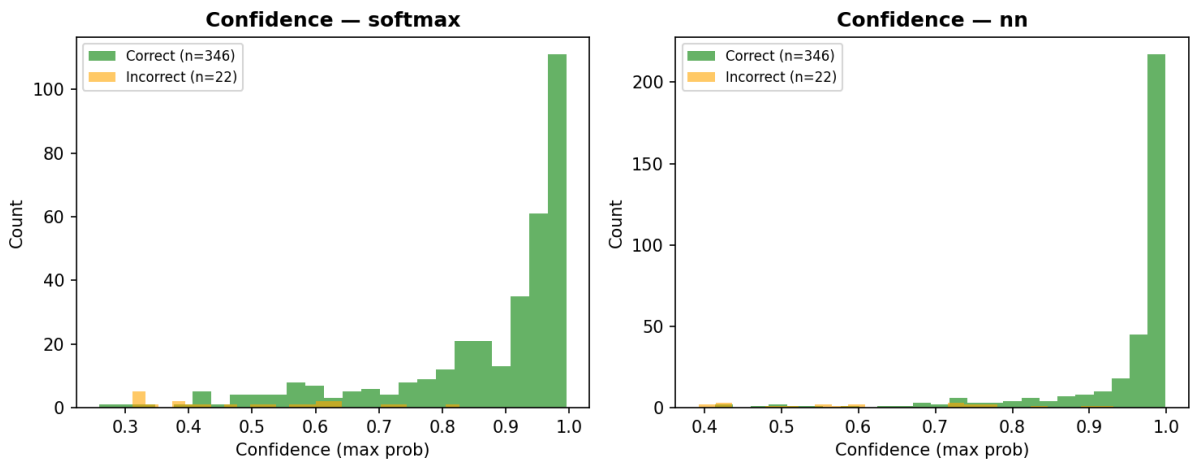

Figure 6: Histogram of predictive confidence for correct and incorrect predictions on the digits benchmark. The neural network produces a larger concentration of highly confident correct predictions and somewhat clearer separation between correct and incorrect cases.

Figure 6 compares the maximum predicted probability for correct and incorrect predictions. For both models, correct predictions tend to cluster at higher confidence than incorrect predictions. However, the neural network produces a stronger concentration near confidence 1.0 for correct cases, which suggests sharper and more decisive classification.

Confidence analysis is useful because two models can have similar accuracy but very different confidence profiles. A model that is both accurate and well-calibrated is generally more useful than one that is merely accurate.

17. Entropy Analysis

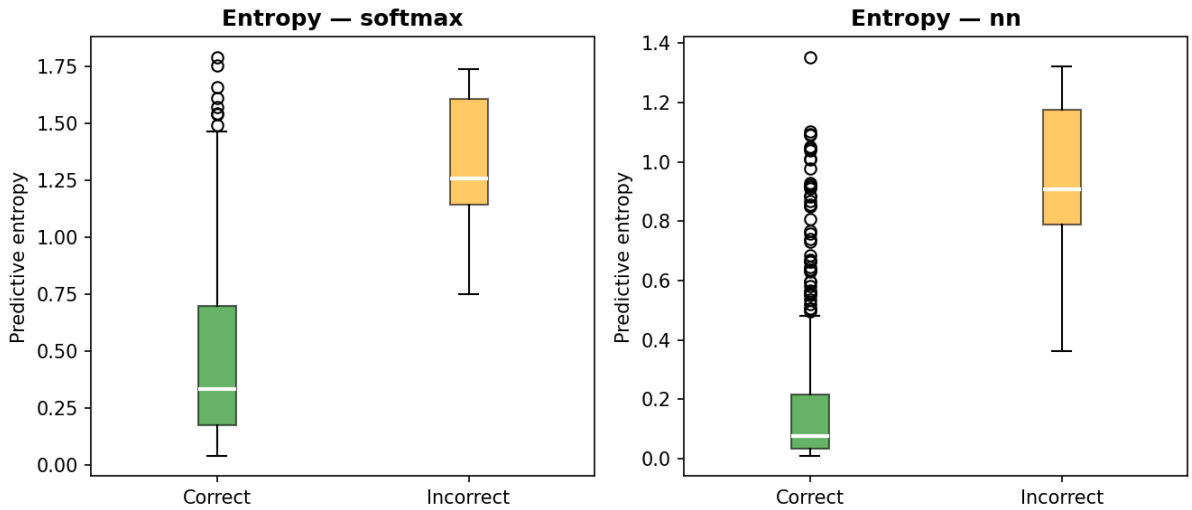

Figure 7: Predictive entropy for correct and incorrect predictions. Correct predictions tend to have lower entropy, while incorrect predictions have higher entropy. The neural network shows especially low entropy for many correct predictions, indicating stronger certainty.

Predictive entropy provides another view of uncertainty. Low entropy means that the model distributes most probability mass to one class, while high entropy means the model is uncertain and spreads probability across several classes.

Figure 7 shows that correct predictions are associated with lower entropy and incorrect predictions with higher entropy for both models. The neural network has a notably lower median entropy on correct cases, which suggests that it is not only more accurate but also more confident when it is right.

18. Reliability Analysis

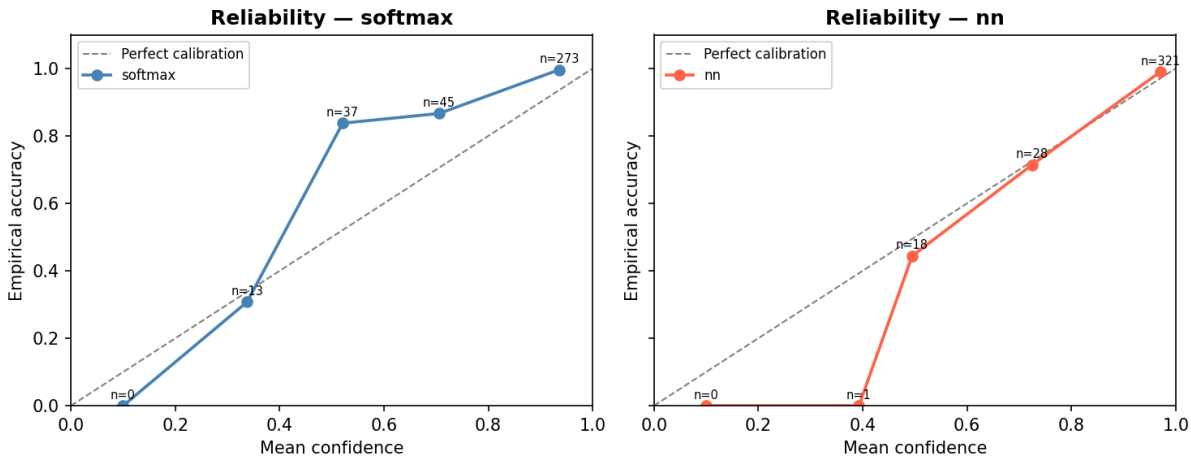

Figure 8: Reliability diagrams for softmax regression and the neural network. The dashed diagonal indicates perfect calibration. The neural network is closer to the ideal line across the high-confidence region, suggesting better probability calibration.

Reliability diagrams compare predicted confidence against empirical accuracy. If a model is perfectly calibrated, then predictions with confidence 0.8 should be correct about 80% of the time, and points should lie near the diagonal.

Figure 8 indicates that the neural network is generally better calibrated, especially in the high-confidence region. This means that its probabilities are more trustworthy as estimates of correctness. Good calibration is valuable in practical applications because it makes the output probabilities more meaningful.

19. Discussion

This project highlights the importance of model selection:

- Linear models are efficient but limited
- Neural networks provide flexibility
- Complexity must match data structure

A central lesson of the project is that model performance must be interpreted in context. It is not enough to say that the neural network performs better overall. On the Gaussian dataset, the neural network does *not* add practical value because the linear classifier is already sufficient. On the moons dataset, however, the neural network is clearly necessary because the boundary is fundamentally non-linear.

The digits benchmark lies in between these two extremes. It is not as visually simple as Gaussian, nor as geometrically obvious as moons. Instead, it tests whether the models can develop useful internal structure over time. The neural network succeeds because its hidden layer can transform the input space in a way that allows the final output layer to separate classes more effectively.

The optimizer and uncertainty analyses extend the discussion further. They show that performance is not only about raw accuracy, but also about how fast the model learns, how confident it is, and whether its probabilities are reliable. These are all part of a mature model evaluation framework.

20. Limitations

This project has several limitations:

- Only one hidden layer was used
- Hyperparameter tuning was limited
- The number of datasets was intentionally small
- Advanced regularization strategies were not explored

These limitations do not invalidate the conclusions, but they do define the scope of the study. A deeper network might perform even better on the digits benchmark, and more extensive tuning could improve both optimization and calibration. However, the point of the capstone is not to maximize leaderboard-style performance, but to understand core machine-learning principles through transparent implementations.

21. Conclusion

The results clearly demonstrate:

Model choice depends on dataset complexity.

- Simple data $\rightarrow$ linear models are sufficient
- Complex data $\rightarrow$ neural networks are necessary

This reinforces the importance of understanding both data geometry and model capacity in machine learning.

More specifically, the project shows that:

- Softmax regression is a strong and interpretable baseline

- One-hidden-layer neural networks can dramatically improve performance on nonlinear tasks
- Capacity, optimizer choice, and predictive reliability all matter when evaluating models

Taken together, the experiments provide a coherent answer to the project's central question. Linear classifiers are appropriate when the data structure is simple and approximately affine. Hidden-layer neural networks become essential when the classes are arranged in more complex ways or when the feature interactions cannot be captured linearly.

22. Future Work

Future improvements may include:

- Exploring deeper neural architectures
- Systematic hyperparameter tuning
- Additional regularization methods
- More detailed repeated-seed statistical analysis
- Broader calibration and uncertainty evaluation

These directions would extend the current study while preserving its main educational value: connecting implementation details, geometry, optimization, and predictive behavior into one coherent machine-learning analysis.